Reviewer Pack — Minimal Diophantine Exponent Bounds DPLL Refutation Time

Entry ae9ddec457ba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 08:36:41 UTC

Minimal Diophantine Exponent Bounds DPLL Refutation Time

Entry ID: `ae9ddec457ba` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-30 08:36:41 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Exponents) **Field B** (complexity object): Boolean Function Complexity: Davis-Putnam Resolution

Statement:

> For every CNF φ with n variables, the minimal diophantine exponent of φ is upper bounded by the DPLL refutation time for φ to the power of log(n). Specifically, if d(φ) is the smallest positive integer such that φ is a solution set modulo d(Z_n), then E[DPLL_refutation_time(φ)] ≤ n^d(φ) * log(n).

Rationale (proposer's reasoning):

> Diophantine exponents describe the complexity of solving polynomial equations over finite fields, which can be linked to propositional satisfiability problems. This conjecture suggests that a lower bound on diophantine complexity could translate into an upper bound on resolution refutation time, revealing a deep connection between number theory and computational complexity.

Taxonomy category: `cg_kw_andreev` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0d12163534d51274`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean of the ratios of n^d(φ) * log(n) to DPLL_refutation_time(φ) across all CNFs is ≤ 3 with a confidence level of 95%.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `title:diophantine exponent AND DPLL refutation` - `min diophantine exponent IN BOOLEAN FUNCTION COMPLEXITY AND DPLL resolution` - `upper bound ON minimal diophantine exponent BY DPLL refutation time`

Top relevant hits considered: - [http://arxiv.org/abs/cs/0209032v3] Complexity Results on DPLL and Resolution - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1609.06986v1] More on Diophantine sextuples - [http://arxiv.org/abs/2107.11134v1] Upper bounds for the uniform simultaneous Diophantine exponents - [http://arxiv.org/abs/1603.03800v3] Diophantine approximation on matrices and Lie groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_cnf(n, seed):
    random.seed(seed)
    cnf = []
    for _ in range(random.randint(5, 10)):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        cnf.append(clause)
    return cnf

def diophantine_exponent(cnf):
    n = len(cnf[0])
    Z_d = list(range(1, 2*n))
    d = 1
    while True:
        if all(any(lit % d == Z_d[var-1] for lit in clause) for clause in cnf):
            return d
        d += 1

def dpll_refutation_time(cnf):
    n = len(cnf[0])
    clauses = [set(clause) for clause in cnf]
    literals = set(range(1, n+1)) | {-i for i in range(1, n+1)}

    def solve(state):
```

```
        if not clauses:
            return True
        literal = next(lit for lit in literals if lit not in state and -lit not in state)
        for val in [True, False]:
            new_state = state.copy()
            new_state[literal] = val
            if all(not clause.intersection(new_state) for clause in clauses):
                if solve(new_state):
                    return True
        return False

    start_time = time.time()
    solve({})
    end_time = time.time()
    return end_time - start_time

def run_trial(seed: int) -> dict:
    n_max = 40
    instances_tested = 0
    metric_value_total = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in range(5, n_max + 1):
        cnf = generate_cnf(n, seed)
        d = diophantine_exponent(cnf)
        refutation_time = dpll_refutation_time(cnf)
        if refutation_time == 0:
            continue
        ratio = (n**d * math.log(n)) / refutation_time
        metric_value_total += ratio
        instances_tested += 1

        if ratio > 3:
            conjecture_holds = False
            counterexample = f"n={n}, d={d}, refutation_time={refutation_time}"

    metric_name = "Ratio of n^d * log(n) to DPLL refutation time"
    metric_value = metric_value_total / instances_tested if instances_tested > 0 else 0.0

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.2:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE insufficient support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9d339317.py", line 96, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9d339317.py", line 66, in run_trial
    d = diophantine_exponent(cnf)
        ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9d339317.py", line 31, in diophantine_e
    if all(any(lit % d == Z_d[var-1] for lit in clause) for clause in cnf):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9d339317.py", line 31, in <genexpr>
    if all(any(lit % d == Z_d[var-1] for lit in clause) for clause in cnf):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9d339317.py", line 31, in <genexpr>
    if all(any(lit % d == Z_d[var-1] for lit in clause) for clause in cnf):
                          ^^^
NameError: name 'var' is not defined. Did you mean: 'vars'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed due to a NameError, which prevented it from producing data necessary to evaluate the conjecture. | next: Debug the code and rerun the test to ensure that all variables are correctly defined before re-evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18651
2	propose	ollama_remote	glm4:latest	0	0	17832
3	propose	ollama_remote	glm4:latest	0	0	16387
4	propose	ollama_remote	glm4:latest	0	0	13205
5	preregistration	ollama_remote	glm4:latest	0	0	8956
6	novelty	ollama_remote	glm4:latest	0	0	8071
7	novelty	ollama_remote	glm4:latest	0	0	9757
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30551
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25310
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15739
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11108
12	judge	ollama_remote	glm4:latest	0	0	8880

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 184447 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/ae9ddec457ba.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ae9ddec457ba.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ae9ddec457ba.tar.gz` (if generated)